

Criteria status — facts only

Mapped to blog.janestreet.com/protocol-emulator-asic-competition

2026-09-15

This note maps the Loom repo to the published Jane Street protocol-emulator ASIC criteria (10 Sep 2026). It records what was measured, what was formally proven, and what has not been run on a pin. It does not discuss trading payloads or architecture tutorials.

Source: <https://blog.janestreet.com/protocol-emulator-asic-competition/> · top tt_um_loom_gpe · tiles 6×4 · clock 50 MHz · license Apache-2.0 · 407 tests collected under pytest tests.

Status in one paragraph. The design is a reprogrammable pin/time ISA, not a hardwired UART/SPI/I2C trio. A 1-state-machine CMOS5L GDS closed at 50 MHz with Magic DRC 0 and a unique LVS match. Current source instantiates a larger 2-SM / 4-slot chip that has been synthesised but not taken to GDS. On the interpreter, all 10 registered TX/RX pairs are exhaustive codecs for bytes 0..255 against independent spec languages (UART hello, UART 8N1, SPI mode-0 with CS, I2C open-drain, JTAG TMS TAP, SWD line-reset, PS/2 host byte, CAN stuffed CRC-15, USB LS NRZI, Ethernet MAC framing). USB and CAN time bits from a helper clock on pin2; that pin is not part of those buses. The ISA is k-induction proven on the 1-SM engine (54 named assertions). UART/SPI/I2C hello pin traces match Amaranth RTL (push-pull I2C, not the open-drain pair). Nothing has been run on an FPGA or a real peer device.

1-SM GDS closed @ 50 MHz DRC 0, LVS match	54 k-induction asserts ISA + FIFO/halt	10 x 256 interp codecs proven all registered pairs	0 FPGA / real-pin bring-up runs
--	---	---	--

1. Contest criteria, as published

Quoted requirements are from the contest page. Status is taken from this tree (plans, tests, LibreLane runs, formal log). "Partial" means the letter of the item is addressed in simulation or in a subset graph, not on silicon and not as a full PHY.

Criterion	What this repo has	Status
Open-source general-purpose protocol emulator ASIC	16-bit pin/wait/delay/shift/fifo/jump/mov ISA. Protocols are TOML graphs in plans/, loaded into imem after tapeout. Engine core is forbidden from naming a protocol (loom check).	Yes — as RTL/graphs
Not a UART + SPI + I2C block trio; new protocols after fabrication	Host two-phase imem write + TX FIFO + run, documented in docs/info.md, tested in tests/test_host_load.py (Amaranth). Reprogram uart→spi→i2c on one interpreter object. Same engine, new words; no resynth.	Yes — in simulation
Start with UART, SPI, and I2C	UART 8N1; SPI mode-0 MOSI/SCK/CS; I2C open-drain START / 8 bits / ACK / STOP. All three proven 0..255 on the interpreter. Push-pull I2C hello graphs still exist and are not the I2C formal case.	Yes — graphs + interp proof
Stretch: low-speed USB and 10 Mbit Ethernet	USB LS: NRZI (0=toggle, 1=hold), stuff after six 1s, SYNC KJKJKJKK, EOP SE0 on D+/D-, timed from pin2. Ethernet: IEEE 802.3 MAC framing (seven 0x55 + 0xD5 SFD + payload) on clock+data. Neither is an analog PHY.	Partial — digital line language
Also named: JTAG, SWD, PS/2, CAN	JTAG: TMS TAP (TLR via ≥5 TMS=1, then Shift-DR). SWD: ≥50 SWDIO=1 clocks then 8 LSB. PS/2: host byte, odd parity, device ACK. CAN: SOF + both-polarity stuff through CRC-15 (0x4599) + ACK 0; proven 0..255 on the interpreter. Helper clock pin2.	Partial — bit-level graphs, not full stacks
FPGA smoke of RTL before ASIC flow	Named in the brief. Not run. No iCEBreaker / TT demo-board log in this tree.	No
Formal methods	Yosys sat k-induction: 1-SM ISA + CRC MOV 7–11 + host CSR writes; ClockedImem; 2-SM next_pc fetch + SM0/SM1 CSR writes. Log: Induction step proven: SUCCESS! (test/gen/formal.log). Does not prove a protocol waveform on RTL.	Yes — ISA, not protocols
Random constrained tests	tests/test_random_jitter.py: UART baud error / edge jitter / runt starts; SPI clock asymmetry, gaps, MOSI noise while SCK low. Decoder is the graph, waveforms are synthesised, not taken from TX.	Yes — UART/SPI interp

Criterion	What this repo has	Status
IHP 130 nm CMOS5L via Tiny Tapeout; start from CMOS5L Verilog template	ttihip-verilog-template CMOS5L. info.yaml tiles 6x4, clock_hz 50000000, CLOCK_PERIOD 20. LibreLane run runs/wokwi completed through GDS.	Yes
Area: 6x4 tiles (~0.7 mm², ~1k cells/tile ≈ 24k cell budget)	Closed GDS (1 SM): 3,499 cells, 56,492 μm². Current 2-SM loom_chip instantiates IHP RM_IHPSG13_2P_256x16_c2_bm_bist for imem. 2-SM+SRAM P&R not closed.	Yes — under budget
Full P&R + timing at a declared clock	1-SM GDS: setup/hold WNS 0 at 50 MHz; slow-corner setup slack +8.986 ns. Magic DRC COUNT 0. Netgen: Circuits match uniquely. KLayout DRC was skipped (RUN_KLAYOUT_DRC false). Slow corner: 8 max-slew, 37 max-fanout violations. 2-SM/4-slot P&R stopped in detailed routing; no GDS.	Yes — 1-SM GDS only
Open source; build in public	Apache-2.0. git remote: github.com/wsb1994/Gremlin-Board.	Yes
Sign-up form / submit by 2027-01-18	Neither form is in the repo. Deadline is in the future.	Open

Table 1. Published contest items versus this tree. Status is not a score.

2. What closed GDS, versus what the source emits now

emit.py writes two engines into src/loom_engine.v: loom_engine (1 SM, 1 slot) and loom_chip (2 SM, 4 slots). The completed LibreLane GDS instantiated loom_engine (623 FFs matches 32x16-bit imem plus SM/FIFO/host state). src/project.v now instantiates loom_chip. Those are different netlists.

CLOSED GDS (runs/wokwi, tt_submission) tt_um_loom_gpe instantiates loom_engine 1 state machine · 1 × 32-word imem 3,499 CMOS5L cells 623 FFs 56,492 sq um ~8% of 6x4 tile area Magic DRC 0 · LVS unique match 50 MHz · slow setup slack +8.99 ns	CURRENT SOURCE (src/project.v) tt_um_loom_gpe instantiates loom_chip 2 state machines · 4 × 32-word imem synth 14,944 cells 2,412 FFs 233,174 sq um still under 6x4 area LibreLane P&R stopped in detailed route no GDS / DRC / LVS for this netlist
---	---

Figure 1. Closed CMOS5L GDS is the 1-SM engine. The 2-SM / 4-slot source has synth numbers only.

k-induction now includes 2-SM fetch and CSR writes (formal_chip). iverilog protocol Hi tests still target loom_engine (1 SM). Dual-SM UART+SPI on disjoint pins is tests/test_infra_desk.py T7. Closed GDS remains the 1-SM engine.

3. Named protocols: what the graphs are

The brief lists UART, SPI, I2C as the start, USB LS and 10 Mbit Ethernet as stretch, and JTAG, SWD, PS/2, CAN as further examples. Every name has a graph pair in plans/. Assembled length is compile_plan word count / 32. The graph is not the full protocol unless the last column says so.

Protocol	Graphs (words)	What the graph actually does	Full protocol?
UART	uart_tx/rx 9/8, uart_8n1_* 9/15, uart_rx_frame 15	8N1 on pin0, LSB first, 8 SM-cycles/bit. Production RX: centre sample, start verify, stop check, frame-error flag on pin1.	Yes — 8N1 byte
SPI	spi_tx 11, spi_rx 8	Mode 0, MOSI=pin0, SCK=pin1, CS=pin2 active-low, MSB first.	Partial — mode-0 byte with CS
I2C hello	i2c_tx 15, i2c_rx 9	Push-pull bit dance on SDA/SCL. Not the formal I2C case.	No — cartoon
I2C open-drain	i2c_od_tx 22, i2c_od_rx 12	gpio_out stays 0; drive-0 is OE. START, 8 MSB, ACK 0, STOP. Golden slave is in tests/test_i2c_opendrain.py, not on the die.	Partial — 1-byte master write
JTAG	jtag_tx 31, jtag_shift 15	TMS TAP: ≥5 TMS=1 (TLR), then Shift-DR, 8 TDI LSB, Update-DR.	Partial — 8-bit Shift-DR, not IDCODE
SWD	swd_tx 16, swd_rx 10	≥50 clocks with SWDIO=1, then 8 LSB on SWCLK. No DPIDR parse.	Partial — line-reset + byte, not DPIDR

Protocol	Graphs (words)	What the graph actually does	Full protocol?
PS/2	ps2_tx 29, ps2_rx 14	Host-clocked byte: start, 8 LSB, odd parity, stop, ACK 0.	Partial — host byte, not a keyboard host
CAN	can_tx 32, can_rx 32	TX: SOF, 8 LSB, both-polarity stuff through CRC-15 (poly 0x4599), ACK 0, one clocked recessive; parks on pull. RX: destuff 5 identical of either polarity, then wait TX pin4 (CRC-phase) before the next SOF. Helper clock pin2.	Partial — stuffed 1-byte data frame, not a CAN node
USB LS	usb_tx 32, usb_rx 32	NRZI (0=toggle, 1=hold), stuff after six 1s, SYNC KJKJKJJK, EOP SE0 on D+/D-. RX samples D+ against previous level on pin3. Helper bit-clock pin2 (not a USB wire).	Partial — NRZI packet, not analog PHY
10 Mbit Ethernet	eth_tx 32, eth_rx 13	Seven 0x55 + 0xD5 SFD + payload on data+clock. Plan text: not 10BASE-T magnetics.	Partial — 802.3 framing, not 10BASE-T

Table 2. Contest-named protocols versus the shipped graphs. “Full protocol?” is relative to the named standard, not to a byte round-trip.

4. Was each path formally proven?

Two different claims live in this repo. They are not interchangeable.

1. ISA k-induction (RTL, protocol-agnostic). generator/loom/formal.py. Yosys sat -tempinduct, 54 named assertions: FIFO occupancy/pointers, halt freezes the SM, run blocks imem writes, and one-step semantics of NOP, JMP (all fields), WAIT, IN, OUT, FIFO pull/push, SET (all fields), MOV dests with payload ≤ 6 (including dest $\wedge = Y$). MOV payloads 7–11 (hidden 15-bit CRC, poly 0x4599) are unconstrained in that proof. Log ends “Induction step proven: SUCCESS!”. The harness ties csr_we=0, so clkdiv stays 1, wrap 0..31, sideset off. It does not mention UART, SPI, or I2C. It does not prove a protocol waveform. It is the 1-SM GraphEngine, not loom_chip.

2. Protocol completeness (interpreter, finite alphabet). generator/loom/formal_proto.py, invoked as loom formal-proto. For each registered TX/RX pair and every byte b in 0..255: TX(b) is a word of the spec language L; RX(spec(b))=b; RX(TX(b))=b; after one byte the SM sits on pull. Spec waves come from generator/loom/line_lang.py (SPECS), not from the TX graph. That is exhaustive model checking of an 8-bit FIFO engine on the *interpreter*, not k-induction of RTL. Measured this session: 10 of 10 pairs complete (see Table 3).

Composition, stated as a composition, not as a solver run. If RTL implements the ISA (k-induction, 1 SM, default CSRs) and a graph is a codec on the interpreter (all 256 bytes), then RTL executing that graph is a codec *under those CSR defaults*. That argument is written in formal_proto.py. It has not been discharged as one formal property. Independent of that, Amaranth RTL pin traces were compared to the interpreter for UART/SPI/I2C hello only.

k-induction · 54 named assertions · 1-SM GraphEngine, default CSRs

Exhaustive 0..255 on the interpreter · 10 of 10 pairs

Amaranth RTL pin traces = interpreter · UART / SPI / I2C hello only

iverilog Hi tests exist on loom_engine · not re-run after USB/CAN rewrite

FPGA pin / USB-UART / flash / EEPROM / TAP · not run

Figure 2. Proof and test layers. A layer does not imply the ones below it.

Path	Exhaustive 0..255 interp	Amaranth RTL pins	Interpreter Hi	k-ind of this waveform
uart_tx / uart_rx	Yes	Yes (Hi trace)	Yes (165 cyc)	No
uart_8n1_tx / uart_8n1_rx	Yes	No	codec 0..255	No

Path	Exhaustive 0..255 interp	Amaranth RTL pins	Interpreter Hi	k-ind of this waveform
spi_tx / spi_rx	Yes	Yes (Hi; graphs now have CS)	Yes (76 cyc)	No
i2c_tx / i2c_rx (push-pull)	No — not the formal case	Yes (Hi trace)	not the PAIRS pair	No
i2c_od_tx / i2c_od_rx	Yes	No	Yes (142 cyc)	No
jtag_tx / jtag_shift	Yes	No	Yes (164 cyc)	No
swd_tx / swd_rx	Yes	No	Yes (385 cyc)	No
ps2_tx / ps2_rx	Yes	No	Yes (166 cyc)	No
can_tx / can_rx	Yes vs can_spec_wave	No	Yes (640 cyc)	No
usb_tx / usb_rx	Yes vs usb_spec_wave	No	Yes (566 cyc)	No
eth_tx / eth_rx	Yes vs eth_spec_wave	No	Yes (554 cyc)	No
ISA step (all 8 opcodes)	n/a	golden snapshots	2048 encodings × 7 stimuli (test exists)	Yes — 1 SM

Table 3. Per-path evidence. “k-ind of this waveform” would mean a solver proof that the RTL pin trace is the protocol. That was not done for any protocol. ISA k-induction is the last row.

Other verification named by the brief	Where	Result
Constrained-random UART/SPI pins	tests/test_random_jitter.py (interpreter)	Yes — those graphs
ASCII waveform expects (JS 2020 style)	tests/expect/uart_hi.txt, spi_hi.txt	Yes — UART/SPI hello
I2C open-drain ACK + stretch vs model	tests/test_i2c_opendrain.py (interpreter)	Yes — vs software slave
Host load two-phase imem + FIFO	tests/test_host_load.py (Amaranth + wrapper model)	Yes — simulation
iverilog Hi on loom_engine	tests/test_verilog_sim.py (every PAIRS name). TB ties gpio_in to gpio_out (no pull-up / OE).	Test exists — not re-run this session
Gate-level cocotb	test/test.py	Partial — idle only (run=0)
Non-default clkdiv / wrap / sideset in k-induction	formal.py comments: covered by interp=RTL tests, not by sat	No — not in the proof
2-SM / 4-slot engine in k-induction or GDS	formal harness GraphEngine(); GDS is loom_engine	No

Table 4. Other verification the contest page asked for by name.

5. Is the simulator usable in a real scenario?

“Simulator” here is the cycle-accurate ISA interpreter (loom.interp) plus, where noted, Amaranth simulation and iverilog on generated Verilog. None of these models pads, cables, analog PHYs, or a real peer device. GPIO inputs are 2-flop synchronised in RTL while running; the interpreter does not model that synchroniser. loom.stream.roundtrip uses a wired-AND pull-up so open-drain ACK (I2C, PS/2) can work in that helper; a raw pin does not.

What it is usable for. Authoring a graph, checking that a TX/RX pair recovers every byte 0..255, locking an ASCII pin trace, and comparing interpreter vs Amaranth vs iverilog before tapeout. That is a real use in a design flow. It is not a protocol analyser, not a stand-in for an FPGA board, and not evidence that a USB-UART, SPI flash, I2C EEPROM, JTAG TAP, or SWD target would lock.

The contest’s stated job for the chip is hardware debugging and reverse engineering: bit-bang what is on the desk, within this die’s rates and 8 GPIO. At 50 MHz with 8 SM-cycles/bit, clkdiv=1 is a 6.25 Mbit/s class line; 115200 8N1 is inside the divider range (clkdiv ≈ 54). That is a cycle-budget fact, not a bring-up result.

Desk scenario	Graph that would be loaded	Simulator evidence	Usable on a real pin today?
UART 115200 8N1 to a USB-UART	uart_8n1_tx / uart_8n1_rx	Codec 0..255 on interp; jitter/run tests on uart_rx_frame and uart_8n1_rx. RTL pin match is the hello pair, not 8n1.	No — never driven off-chip
SPI mode-0 flash JEDEC ID (0x9F)	spi_tx / spi_rx	Byte codec 0..255 with CS. No flash in the formal suite.	No — no flash
I2C EEPROM read, addr 0x50	i2c_od_tx / i2c_od_rx	Codec 0..255. Master write 0xA0 then 0x48 with ACK and stretch vs a software slave.	No — model only
JTAG TAP IDCODE	jtag_tx / jtag_shift	8-bit Shift-DR after TLR, codec 0..255. A 32-bit IDCODE sequence is not the graph.	No — cannot IDCODE
SWD line reset + DPIDR	swd_tx / swd_rx	Line-reset (≥ 50 SWDIO=1 clocks) + 8 data bits, codec 0..255. No DPIDR parse.	No — cannot DPIDR
PS/2 keyboard byte	ps2_tx / ps2_rx	Host byte with odd parity and ACK, codec 0..255 on the interpreter.	No — never a keyboard
Low-speed USB device/host	usb_tx / usb_rx	NRZI+stuff+SYNC+EOP codec 0..255 on interp. RX/TX use pin2 as a bit-clock a USB cable does not carry. No analog PHY.	No — not a USB PHY; extra clock pin
10BASE-T Ethernet	eth_tx / eth_rx	Preamble/SFD/payload codec 0..255 on clock+data. Not Manchester, not magnetics.	No — not 10BASE-T
CAN 2.0 at 125 kbit	can_tx / can_rx	Stuffed CRC-15 codec 0..255 on interp; Hi and PNG SHA-256 round-trip. Helper clock pin2. No analog PHY, no arbitration.	No — not a CAN transceiver
Reprogram after fab (halt, load a new graph, run)	any pair, same engine	Interpreter reprogram test; host-load Amaranth test. Dual-SM UART TX+RX on internal GPIO in interp.	No — no board

Table 5. Real-scenario usability. “No” means this tree has not talked to that device, and/or the graph is not the protocol that device speaks.

After fabrication of the closed 1-SM GDS, a host could in principle load uart_8n1 or spi or i2c_od and bit-bang those byte-level contracts on 8 GPIO, at rates the 50 MHz divider can hit. That is an architectural claim plus simulation. It has not been demonstrated. The 2-SM / 4-slot source would add concurrent graphs; that netlist has no GDS.

6. Remaining contest items

Item	Why it is still open
Sign-up form on the contest page	Required to receive the submission link. Not a commit.
Final submission by 2027-01-18	Form is not on the page yet; nothing submitted.
FPGA smoke of UART Hi on a pin	Named in the brief. No board log in this tree.
P&R / GDS of the 2-SM / 4-slot netlist	Synth fits 6×4; LibreLane stopped in detailed routing.
KLayout DRC	Skipped on the closed 1-SM run (RUN_KLAYOUT_DRC false). Magic DRC was 0.
Gate-level UART Hi	cocotb GATES=yes presently asserts only that run=0 after reset.
k-induction of loom_chip, clkdiv, wrap, sideset, CRC MOV	Harness is 1-SM defaults; CRC MOV unconstrained.
End-to-end formal of a protocol waveform on RTL	Not done for any named protocol.
CAN analog PHY / multi-node arbitration	Interpreter codec is a 1-byte stuffed data frame on GPIO+clock.
Any T1–T8 desk test (USB-UART, W25, EEPROM, TAP, DPIDR)	Simulation and models only.
8×4 tile	Contest may offer it later. CMOS5L tooling in this tree defines 6×4.

Table 6. Open items relative to the published brief.

Bottom line against the brief. Architecture matches: a PIO-class reprogrammable pin/time engine, open source, on the CMOS5L 6×4 template, with a timing-closed 1-SM GDS. UART, SPI (mode-0 with CS), and open-drain I2C graphs exist and are exhaustive codecs on the interpreter. Stretch USB and Ethernet are digital line languages (NRZI packet; 802.3 framing on clock+data), not analog PHYs, and those codecs are proven 0..255 on the interpreter. JTAG/SWD/PS/2/CAN graphs implement the named bit-level mechanics in 32 words and are proven the same way. Formal methods were applied to the ISA, not to each protocol path on RTL. The simulator is usable as a graph lab; it is not evidence of a real-desk protocol emulator until a pin test exists.

Rebuild: PYTHONPATH=generator python scripts/build_exec_summary.py. Evidence files: test/gen/formal.log, runs/wokwi (GDS), src/runs/loom-chip (2-SM synth), generator/loom/formal.py, generator/loom/formal_proto.py, docs/info.md.